



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES



Despliegue remoto en PaaS

DESARROLLO FULLSTACK I
DSY1103



• Contenidos

- PaaS
- Docker

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern lighting. In the foreground, there is a desk with a coffee cup and some papers.

01

**Despliegue remoto
en PaaS (Render /
Railway /
AlwaysData)**

• Qué es PaaS y por qué es relevante para microservicios

PaaS (Platform as a Service) es un modelo de computación en la nube donde la plataforma ofrece:

- infraestructura (servidores, redes, seguridad)
- ambiente de ejecución (runtime, JDK, Node, Python, etc.)
- herramientas de despliegue continuo
- administración de entornos

y el desarrollador solo se preocupa de su código.

En microservicios PaaS se usa para:

- desplegar servicios individualmente
- centralizar variables de entorno
- habilitar escalabilidad horizontal
- probar microservicios con clientes reales
- separar entornos (dev / staging / prod)

PaaS es el paso natural luego de desarrollar y testear localmente.

• Qué es PaaS y por qué es relevante para microservicios

Aspecto	Render	Railway	AlwaysData
Tipo	PaaS moderno	PaaS orientado a desarrolladores	Hosting general + PaaS
Integración GitHub	Excelente	Excelente	Media
Base de datos	Muy buena	Muy buena	Limitada
Facilidad	Muy fácil	Muy fácil	Media
Ideal para	Microservicios, API Gateway	Microservices + DB	Servicios pequeños

• Consideraciones técnicas para desplegar microservicios en PaaS

Requisitos para que un microservicio Spring Boot funcione en la nube

- Debe correr en un solo puerto, usualmente proporcionado por la plataforma. Ejemplo: Railway usa variable `${PORT}`.
- Debe iniciar sin dependencias que no existan en la nube:
 - Eureka opcional
 - Base de datos local no permitida
- Debe empaquetarse como JAR ejecutable (Spring Boot ya lo hace).
- Debe exponer sus endpoints públicamente.

Variables de entorno esenciales

Variable	Uso
<code>PORT</code>	puerto asignado por la plataforma
<code>SPRING_PROFILES_ACTIVE</code>	perfil usado: remote/dev
<code>EUREKA_CLIENT_SERVICEURL_DEFAULTZONE</code>	URL del servidor Eureka
<code>DB_HOST</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	base de datos remota (si aplica)

• Ventajas de YAML en Spring Cloud Gateway

Perfiles para local y remoto

```
application-dev.yml  
application-remote.yml
```

Con configuraciones separadas:

- dev → localhost, puertos fijos
- remote → puertos dinámicos y URLs públicas

Esto permite que el microservicio se adapte dinámicamente al entorno



02

Docker

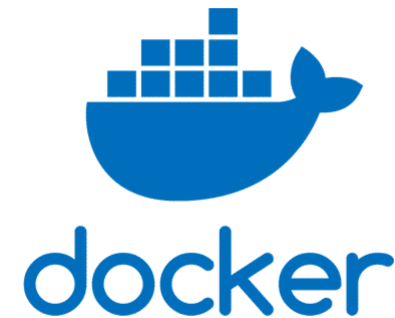
• ¿Qué es Docker?

Docker es una plataforma de contenedores que permite empaquetar aplicaciones junto con todas sus dependencias en imágenes reproducibles, garantizando que funcionen igual en cualquier entorno.

Un contenedor es una “mini-máquina” que contiene:

- Código
- Runtime
- Dependencias
- Configuraciones

No contiene un SO completo → por eso es rápido, liviano y replicable.



[Docker](#)

• ¿Por qué usar Docker en microservicios?

Beneficio	Explicación
Homogeneidad	Tu microservicio corre igual en cualquier máquina u OS
Escalabilidad	Levantar 5 copias del mismo microservicio es trivial
Aislamiento	Cada microservicio corre independiente
Portabilidad	La misma imagen funciona en Render, Railway, AWS, etc.
DevOps-ready	Es la base para CI/CD, Kubernetes, ECS, etc.

Conceptos clave

- **Dockerfile**: archivo que define cómo construir una imagen.
- **Imagen**: conjunto de capas que contiene la app.
- **Contenedor**: instancia en ejecución de una imagen.
- **Volumen**: almacenamiento persistente.
- **Redes**: comunicación entre contenedores.
- **Docker Compose**: herramienta para orquestar múltiples contenedores.

• Buenas prácticas profesionales para Docker

- Usar imágenes base Alpine (más ligeras).
- Separar perfiles: local, remote, docker.
- Nunca meter secretos en el Dockerfile.
- Leer PORT desde variables de entorno.
- Mantener los logs dentro del contenedor simples.
- Subir imágenes optimizadas a Docker Hub.
- No usar volúmenes innecesarios en microservicios.
- Utilizar docker compose para entornos multi-servicio.

[Docker](#)

DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.